In theory this will make subsequent calls to the same accessor perform faster. This may or may not be worth the trouble, depending on the access patterns to your object.

Also notice how we had to jump through some hoops to define the method. We want to define the method only for the current object. This means we have to put the method into the object's singleton class. Ordinarily, we could do that via a Ruby method called define_singleton_method, but that method is defined in Object, not BasicObject, so it's not available to us here.

So we hack it ourselves. We can do that using instance_exec and define_method. But that means we have to use the class << self trick to get the object's singleton class. Through an interesting implementation subtlety, define_method will always define an instance method, independent of whether it is invoked via instance_exec or class_exec.

However, this code reveals a dark underbelly of using method_missing and BasicObject:

metaprogramming/open_struct_flaw.rb

```
obj = MyOpenStruct.new(name: "Dave")
obj.address = "Texas"

o1 = obj.dup
o1.name = "Mike"
o1.address = "Colorado"
```

produces:

```
code/metaprogramming/open_struct_flaw.rb:41:in `<main>': undefined method `name='
for nil:NilClass (NoMethodError)

o1.name = "Mike"
  ^^^^^^^
```

The dup method is not defined by BasicObject; it appears in class Object. So when we called dup, it was picked up by our method_missing handler, and we just returned nil (because we don't have an attribute called dup yet). We could fix this so that it at least reports an error:

```
def method_missing(name, *args, &block)
  if name[-1] == "="
    # as before...
  else
    super unless @values.has_key? name
    # as before...
  end
end
```

This class now reports an error if we call dup (or any other method) on it. However, we still can't dup or clone it (or inspect, convert to a string, and so on). Although BasicObject seems like a natural fit for method_missing, you may find it to be more trouble than it's worth.

Using method_missing as a Filter

As the previous example showed, method_missing has some drawbacks if you use it to intercept all calls. It is probably better to use it to recognize certain patterns of call, passing on those it doesn't recognize to its parent class to handle.

An example of this is the dynamic finder facility used to be found in the Ruby on Rails ActiveRecord module. ActiveRecord is the object-relational library in Rails—it allows you to access relational databases as if they were object stores. One particular feature allows you